

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА
ФРАНКА

Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи №1

“ОСНОВИ НЕРЕЛЯЦІЙНИХ БАЗ ДАНИХ NOSQL”

Виконав:

Студент 3 курсу

групи Фел-33

Абдулханов Абдул-Рахім

Викладач:

ас. Жишкович А. В.

Львів-2026

Мета роботи: Ознайомитися з основами нереляційних баз даних NoSQL, з'ясувати причини їхньої появи та ключові відмінності від традиційних реляційних баз даних (SQL). Дослідити фундаментальні принципи розподілених систем: теорему CAP, порівняння транзакційних моделей ACID та BASE, види масштабування (Scale-Up та Scale-Out). Навчитися обирати відповідний тип СУБД під конкретні практичні задачі та зрозуміти концепцію багатоваріантної персистентності (Polyglot Persistence).

Теоретичні відомості:

Багато років стандартом для збереження інформації були реляційні бази даних (SQL: PostgreSQL, MySQL, Oracle). Вони зберігають дані у вигляді чітких таблиць зі зв'язками між ними і гарантують надійність кожної операції за стандартом ACID. Проте з появою інтернету, смартфонів і соцмереж обсяг інформації виріс у мільйони разів, виникло поняття Великих даних (Big Data).

Великі дані описуються правилом 3V:

1. Volume (величезний обсяг): терабайти і петабайти інформації, які вже не поміщаються на один комп'ютер;
2. Velocity (висока швидкість): мільйони запитів на секунду (лайки, повідомлення, GPS-координати);
3. Variety (різноманітність форматів): текст, фото, JSON, аудіо, які важко втиснути у фіксовані таблиці.

Традиційні бази SQL не змогли ефективно впоратися з цим навантаженням, оскільки їх важко розподілити на сотні звичайних серверів. Як відповідь виник рух NoSQL ('Not Only SQL' - не лише SQL). Термін NoSQL уперше запропонував Карло Строщі у 1998 р., а в сучасному значенні його відродив Йохан Оскарссон у 2009 р. Бази NoSQL відмовилися від жорстких таблиць і важких операцій JOIN на користь простих і швидких структур (документи, пари ключ-значення, колонки, графи та векторні сховища). Замість суворого стандарту ACID у NoSQL використовують гнучкішу модель BASE (доступність і узгодженість із часом), а поведінка системи при збоях пояснюється теоремою CAP (сформульованою Еріком Брюером у 2000 р. та доведеною Сетом Гілбертом і Ненсі Лінч у 2002 р.).

Виконання роботи

1. Порівняння реляційних (SQL) та нереляційних (NoSQL) баз даних:

Склав порівняльну таблицю за 5 основними параметрами, визначеними в завданні:

Параметр порівняння	Реляційні БД (SQL / RDBMS)	Нереляційні БД (NoSQL)
1. Структура даних	Строго таблична: рядки та фіксовані стовпці. Дані нормалізуються, зв'язки будуються через первинні й зовнішні ключі (PK / FK).	Агрегатна: самодостатні JSON/BSON документи, пари ключ-значення, колонкові сімейства (Wide-Column) або графи (вершини і ребра).
2. Гнучкість схеми	Жорстка схема (Schema-on-Write). Щоб додати нове поле, потрібно виконувати важку міграцію ALTER TABLE, яка блокує таблицю.	Динамічна схема (Schema-on-Read / Schemaless). Кожен запис чи документ може мати свій набір полів без зміни всієї бази.
3. Масштабування	Переважно вертикальне (Scale-Up): нарощування потужності одного сервера. Горизонтальний шардинг дуже дорогий і складний.	Природне горизонтальне (Scale-Out): кластер масштабується простим додаванням звичайних недорогих серверів (автошардинг).
4. Наявність JOIN	Повна вбудована підтримка операцій JOIN між таблицями. На великих базах важкі JOIN суттєво гальмують роботу.	Операції JOIN або повністю відсутні, або мінімізовані. Усі пов'язані дані вкладаються в один документ денормалізовано.
5. Підтримка транзакцій (ACID)	Повна підтримка ACID 'з коробки'. Гарантується 100% точність і надійність навіть при збоях живлення (ідеально для грошей).	Орієнтовані на модель BASE (Eventual Consistency). Атомарність гарантується на рівні одного документа, а не всієї бази.

Обґрунтування вибору бази даних під конкретні задачі:

- Для системи бронювання авіаквитків -> обов'язково обираємо реляційну базу SQL (наприклад, PostgreSQL або Oracle).

Пояснення: У літаку кожне місце суворо обмежене. Якщо двоє пасажирів одночасно захочуть купити одне й те саме крісло, реляційна база завдяки транзакціям ACID надійно заблокує рядок і продасть квиток лише одному, а другому видасть помилку. Овербукінг та стан гонитви тут неприпустимі, фінансова точність стоїть на першому місці.

- Для каталогу товарів інтернет-магазину -> обираємо документо-орієнтовану

базу NoSQL (наприклад, MongoDB).

Пояснення: У сучасному маркетплейсі продаються абсолютно різні товари. Для смартфона потрібні характеристики процесора й пам'яті, для взуття - розмір і матеріал, а для книги - автор і кількість сторінок. У NoSQL кожен товар зберігається як окремий JSON-документ із будь-якими потрібними полями. Не треба переробляти всю базу, коли з'являється новий незвичайний товар.

2. Передумови виникнення NoSQL та приклади реальних сервісів:

Перехід від SQL до NoSQL відбувся через 3 головні причини (правило 3V Великих даних):

1) Величезні обсяги даних (Volume), які не вміщаються на один сервер:

- Суть: Реляційну базу найпростіше тримати на одному потужному комп'ютері. Але коли даних стає сотні терабайт, навіть найдорожчий сервер переповнюється.

- Приклад сервісу: Netflix. Сервіс щодня збирає гігабайти історії переглядів від сотень мільйонів користувачів. Зберігати таку купу даних на одному сервері неможливо, тому Netflix використовує NoSQL базу Apache Cassandra, яка розподіляє інформацію по тисячах комп'ютерів.

2) Шалена швидкість надходження запитів (Velocity):

- Суть: Коли мільйони людей щомиті щось пишуть або лайкають, звичайна база блокує таблиці для перевірки і зависає.

- Приклад сервісу: Instagram. Під час трансляцій чи публікацій популярних блогерів сипляться мільйони лайків на секунду. Instagram записує їх у швидкі сховища NoSQL (Cassandra, RocksDB, Redis), які просто зберігають значення в оперативну пам'ять без складних перевірок.

3) Різноманітність та мінливість структури даних (Variety):

- Суть: Користувацькі профілі, пости та картки товарів постійно змінюються. У SQL зміна колонок таблиці (ALTER TABLE) може зупинити сайт на кілька годин.

- Приклад сервісу: Amazon. На маркетплейсі мільйони товарів різних категорій. База Amazon DynamoDB дозволяє кожному товару мати унікальні властивості без попереднього створення жорстких таблиць, що дозволяє оновлювати магазин без зупинки його роботи.

3. Теорема CAP простими словами та аналіз життєвих сценаріїв:

Суть теореми CAP:

Уявімо, що наша база даних працює на кількох серверах, з'єднаних між собою

мережею. Теорема стверджує, що одночасно можна гарантувати лише 2 з 3 речей:

- С (Consistency - узгодженість): на всіх серверах записані абсолютно однакові свіжі дані;
- А (Availability - доступність): сервери завжди дають відповідь і не видають помилок;
- Р (Partition Tolerance - стійкість до розриву зв'язку): система продовжує працювати, навіть якщо кабель між серверами перебито.

Оскільки в реальному житті інтернет та мережа завжди можуть зламатися, властивість Р є обов'язковою! Тому архітектор завжди обирає між двома варіантами: СР або АР.

Аналіз двох ситуацій:

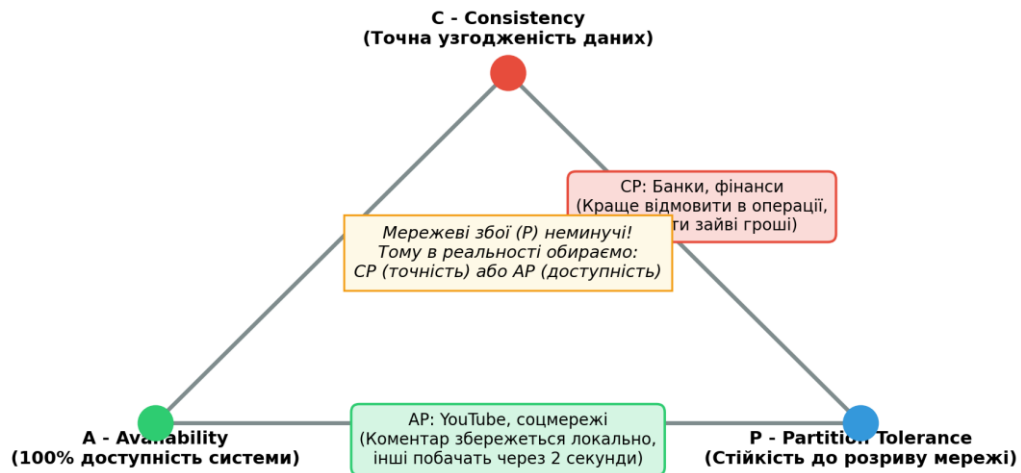
а) Списання коштів у банку -> обираємо модель СР (Узгодженість понад усе): Якщо зв'язок між сервером банку в Києві та Львові обірвався, система повинна **ЗАБЛОКУВАТИ** операцію і видати помилку 'Спробуйте пізніше'. Якщо ми оберемо доступність (АР), клієнт зможе зняти свої останні 1000 грн у Києві і одночасно переказати їх комусь у Львові (бо львівський сервер ще не знає про зняття). Для банку втрата грошей гірша, ніж тимчасова відмова в обслуговуванні.

б) Публікація коментаря на YouTube -> обираємо модель АР (Доступність понад усе):

Якщо між дата-центрами впав зв'язок, YouTube все одно приймає коментар у користувача і показує статус 'Опубліковано'. Хай навіть користувачі в іншій країні побачать цей коментар із затримкою в кілька секунд після відновлення

зв'язку - це нікому не зашкодить. Для YouTube головне, щоб сайт ніколи не видавав помилок і залишався швидким.

Теорема CAP (Брюера): вибір між CP та AP при збоях мережі



4. Порівняння моделей ACID та BASE. Концепція Eventual Consistency:

Для наочності звів основні відмінності між класичними транзакціями ACID та гнучким підходом BASE у таблицю:

Характеристика	Модель ACID (традиційний SQL)	Модель BASE (розподілений NoSQL)
Основна філософія	Песимістична: надійність і точність понад усе. Краще заблокувати запис, ніж зберегти неточні дані.	Оптимістична: доступність і швидкість понад усе. Сервіс має працювати без відмов навіть при збоях.
Перша літера	Atomicity (Атомарність): операція або виконується вся повністю, або повністю скасовується.	Basically Available (Базова доступність): система відповідає завжди, навіть якщо частина серверів впала.
Друга літера	Consistency (Узгодженість): дані завжди відповідають усім правилам і зовнішнім ключам.	Soft state (Гнучкий стан): стан даних може змінюватися сам по собі через фонову реплікацію.

Третя літера	Isolation (Ізольованість): паралельні транзакції не бачать проміжних станів одна одної.	Eventual consistency (Кінцева узгодженість): через короткий час (частки секунди) всі копії стануть однаковими.
Четверта літера	Durability (Довговічність): якщо транзакція успішна, дані гарантовано збережені на диску.	- (покладається на множинну реплікацію на інші сервери)

Пояснення Eventual Consistency (узгодженості в кінцевому підсумку):

Це принцип, за яким система гарантує, що якщо нові зміни не надходять, то через деякий час усі копії даних на всіх серверах стануть однаковими.

Чому для Facebook це нормально:

Коли ви викладаєте новий пост, він спочатку записується на найближчий до вас сервер. Ваші друзі, підключені до інших серверів, можуть побачити цей пост на 2-3 секунди пізніше, поки база розсилає оновлення. Це нормально, бо від затримки у 2 секунди ніхто не постраждає, зате сайт працює блискавично і ніколи не зависає.

Чому це неприпустимо для банківського рахунку:

Якщо оновлення балансу затримається на 3 секунди, людина зможе двічі витратити одні й ті самі гроші в різних магазинах. У банку кожна транзакція має застосовуватися миттєво і синхронно на всіх серверах (за правилами ACID).

5. Масштабування: Scale-Up проти Scale-Out:

Різниця між видами масштабування:

1. Вертикальне масштабування (Scale-Up):

Це як спроба перевезти більше вантажу, купуючи все більшу й дорожчу вантажівку. Ми беремо один сервер і докуповуємо до нього дорожчий процесор, більше оперативної пам'яті (RAM) та швидші SSD. Мінуси: існує фізична межа (не можна вставити більше пам'яті, ніж підтримує материнська плата) і шалена вартість топ-заліза.

2. Горизонтальне масштабування (Scale-Out):

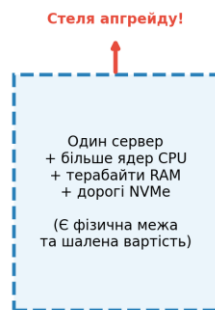
Це як найняти 10 звичайних мікроавтобусів замість одного гігантського

супертягача. Ми об'єднуємо в одну мережу багато звичайних, недорогих серверів. Якщо навантаження зросло - просто докуповуємо ще 2-3 звичайні комп'ютери.

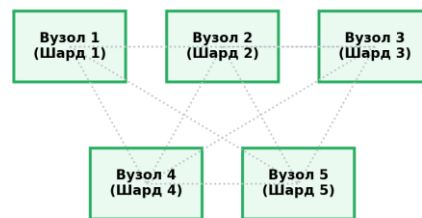
Чому NoSQL бази створені саме для Scale-Out:

У реляційних базах дані розбиті на нормалізовані таблицьки зі зв'язками, тому розділити одну таблицю на 10 різних комп'ютерів дуже складно (щоб зробити JOIN, серверам доведеться постійно пересилати терабайти даних мережею туди-назад). А в NoSQL кожен документ зберігається цілком самостійно. База просто ділить документи на шматки (це називається шардинг) і розкидає їх по різних серверах кластера, що дозволяє рости без обмежень.

Вертикальне (Scale-Up / SQL)
Один дедалі потужніший сервер



Горизонтальне (Scale-Out / NoSQL)
Багато звичайних серверів у кластері



Легко масштабується додаванням нових вузлів

6. Переваги та недоліки баз даних NoSQL:

Підсумував головні плюси та мінуси технології NoSQL у вигляді зрозумілої таблиці:

Переваги NoSQL	Пояснення простими словами	Недоліки NoSQL	Пояснення простими словами
Легке масштабування (Scale-Out)	Якщо серверів не вистачає, просто докуповуємо звичайні дешеві вузли і вмикаємо в кластер.	Слабші транзакції між сутностями	Важко провести одну безпечну транзакцію, яка одночасно змінює кілька різних колекцій.
Гнучка схема (Schemaless)	Можна додавати нові поля до товарів без зупинки роботи сайту та без міграцій таблиць.	Дублювання даних (денормалізація)	Ім'я автора чи покупця копіюється в кожен документ; при зміні імені треба оновити багато місць.
Висока швидкість читання	Всі дані замовлення лежать в одному JSON-документі, зчитуються за один запит без повільних JOIN.	Немає єдиного стандарту мови	У SQL всюди звичні SELECT/INSERT, а в кожній NoSQL свій власний синтаксис запитів.

Висока відмовостійкість	Дані дублюються на кілька серверів (репліки); якщо один сервер згорів, сайт продовжує працювати.	Гірше для складної звітності	Для складної аналітики з багатьма умовами та перехресними таблицями SQL все ще набагато зручніший.
-------------------------	--	------------------------------	--

7. Аналіз ринку DB-Engines та концепція Polyglot Persistence:

Аналіз тенденцій рейтингу DB-Engines за останні роки:

1. MongoDB стабільно утримує перше місце серед нереляційних СУБД завдяки своїй універсальності (документи у форматі BSON/JSON);
2. Redis є абсолютним лідером серед кеш-систем типу ключ-значення, оскільки тримає всі дані в оперативній пам'яті та відповідає за мікросекунди;
3. Головний тренд останніх років - стрімке зростання векторних баз даних (Vector DB: Pinecone, Milvus, Qdrant). Вони зберігають математичні вектори (ембедінги) для систем штучного інтелекту, чат-ботів на базі LLM та пошуку за змістом;
4. Класичні SQL бази (Oracle, MySQL, PostgreSQL) все одно залишаються на вершині загального рейтингу, тому що світ не відмовився від SQL, а прийшов до комбінованого використання обох підходів.

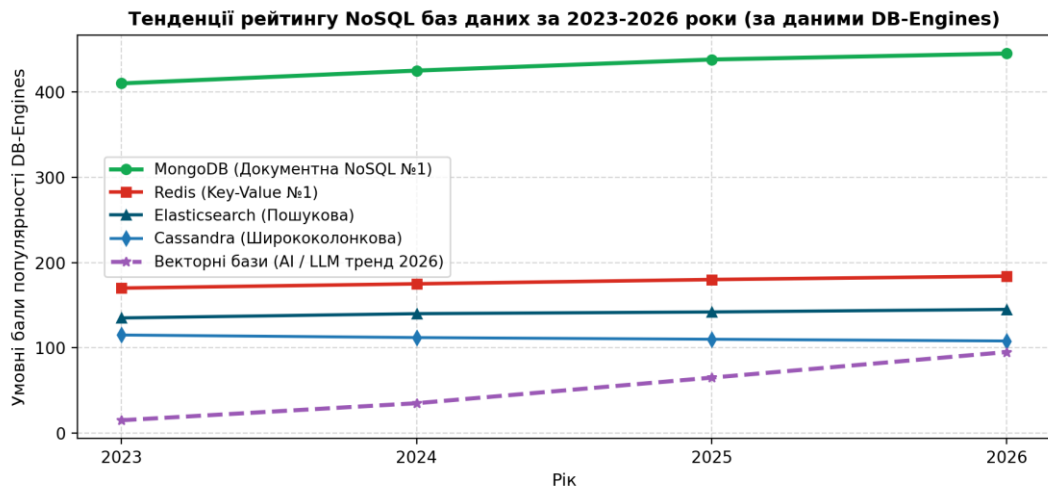
Концепція Polyglot Persistence (багатоваріантна персистентність):

Суть: у великому проєкті не існує однієї 'ідеальної' бази даних на всі випадки життя. Різні частини системи вирішують різні задачі, тому архітектори поєднують кілька різних баз даних в одному додатку.

Приклад сервісу таксі (Uber / Uklon):

- Redis (Key-Value) -> зберігає поточні GPS-координати водіїв та активні сесії пасажирів, бо дані оновлюються щосекунди і потрібна миттєва швидкість у пам'яті;
- MongoDB (Document) -> зберігає профіль користувача, відгуки та деталі поїздки зі змінною інформацією;
- Neo4j (Graph) -> розраховує найкоротші маршрути на карті міста та аналізує дорожні зв'язки;

- PostgreSQL (SQL) -> веде суворий фінансовий облік поїздок, списання коштів з карток та виплати водіям за правилами ACID.



8. Відповіді на питання для самоконтролю (розділ 1.5):

1. Причини створення NoSQL: Необхідність обробляти Величезні дані (Big Data), висока швидкість запису, відсутність жорсткої схеми та неможливість дешево масштабувати реляційні бази на сотні серверів.

2. Класифікація баз даних за моделлю: Реляційні (таблиці зі зв'язками) та Нереляційні (документні, ключ-значення, ширококолонкові, графові, векторні).

3. Що таке агрегат: Це самодостатня сукупність пов'язаних даних (наприклад, замовлення разом з усіма товарами в ньому), яка зберігається і читається як одне ціле. NoSQL називають агрегатними, тому що вони оперують цілими готовими документами, а не розкидають дані по різних таблицях.

4. Переваги NoSQL у HighLoad: Величезна пропускна здатність на запис, робота без блокувань таблиць, швидке читання без JOIN та легке підключення нових серверів.

5. Теорема CAP: При неминучих збоях мережі (P) система може гарантувати або точну узгодженість (CP), або безперервну доступність (AP). Одночасно мати і C, і A при розриві мережі неможливо.

6. Масштабування, шардинг та партиціювання: Scale-Up - збільшення потужності одного сервера, Scale-Out - збільшення кількості серверів. Шардинг - це розбиття великої бази даних на окремі частини (шарди) і розселення їх по різних серверах за певним ключем (наприклад, за першою літерою прізвища чи ID).

7. Реплікація даних: Створення однакових копій бази на кількох серверах. Буває Master-Slave (один головний на запис, інші на читання) або Multi-Master (запис на будь-який вузол). Потрібна для захисту від збоїв та прискорення читання.

8. BASE вимоги: Базова доступність (Basically Available), гнучкий стан (Soft state) та узгодженість із часом (Eventual consistency). На відміну від суворого ACID, BASE ставить швидкість і роботу сервісу вище за миттєву точність.

9. Чому NoSQL відмовляються від повного ACID: Бо синхронізація транзакцій між сотнями серверів у різних країнах вимагає постійних блокувань і робить систему надзвичайно повільною.

10. Суть Polyglot Persistence: Використання кількох різних баз даних в одній системі під конкретні потреби (наприклад, Redis для кешу, MongoDB для каталогу, Postgres для грошей).

11. Основні типи NoSQL баз даних:

- Ключ-значення (Redis) - для кешу та сесій;
- Документні (MongoDB) - для каталогів товарів, профілів та контенту;
- Ширококолонкові (Cassandra, HBase) - для телеметрії, логів і часових рядів;
- Графові (Neo4j) - для друзів у соцмережах та пошуку маршрутів;
- Векторні (Pinecone, Milvus) - для нейромереж, ембедінгів та штучного інтелекту.

12. Сфери застосування NoSQL: Соціальні мережі (пости, лайки), мобільні додатки, маркетплейси (каталоги товарів), інтернет речей IoT (збір показників датчиків), онлайн-ігри та рекомендаційні системи.

Висновок:

У ході виконання лабораторної роботи було детально вивчено фундаментальні принципи та передумови виникнення нереляційних баз даних NoSQL.

З'ясовано, що рух NoSQL виник не як повна заміна реляційним СУБД, а як відповідь на виклики Великих даних (Big Data 3V: Volume, Velocity, Variety), коли реляційні таблиці вперлися у фізичну стелю вертикального масштабування.

На основі аналізу сформульовано такі ключові висновки:

1. Відмінності SQL та NoSQL: Реляційні бази забезпечують сувору фінансову точність (ACID) та нормалізовані таблиці, але важко масштабуються. NoSQL відмовляються від операцій JOIN на користь денормалізованих документів та

природного горизонтального масштабування (Scale-Out) на сотні вузлів.

2. Теорема CAP та компроміси: Оскільки збої мережі в розподіленому середовищі є фізично неминучими (Partition Tolerance), інженер завжди обирає між гарантією точності даних (модель CP, як у банківських системах) та безперервною доступністю сервісу (модель AP, як у соцмережах та YouTube).

3. Модель BASE та Eventual Consistency: Замість блокувань бази даних NoSQL використовують узгодженість у кінцевому підсумку, що дозволяє обслуговувати мільйони запитів на секунду без падіння системи.

4. Сучасний стандарт Polyglot Persistence: Сучасні великі проекти не обирають одну базу 'на все життя', а поєднують кілька технологій одночасно (Redis для швидкого кешу, MongoDB для товарів, Neo4j для зв'язків та PostgreSQL для платежів).

Отримані знання дозволяють чітко розуміти внутрішню архітектуру сучасних СУБД, аргументовано вибирати стек збереження даних під реальні бізнес-вимоги та успішно проектувати високонавантажені інформаційні системи.